

Exceptions

Exceptions: Concepts clés

- **Générer/lever une exception**
- **Prise en charge d'une exception**
- **Exceptions non prises en charge**
- **Objets exception: (information sur l'exception)**

Exceptions communes

IndexError: quand un entier n'est pas dans l'intervalle attendu:

```
>>> x = [2,3,4]
```

```
>>> x[3] provoque une IndexError
```

ValueError: quand un objet est d'un type adéquat mais dont la valeur n'est pas appropriée:

```
>>> int("ballon") provoque une ValueError
```

KeyError: une recherche dans un dictionnaire avec une clé inexistante:

```
>>> codes = dict.gb = 44, us = 1, no = 47, fr = 33, es = 34)
```

```
>>> codes['de'] provoque une KeyError
```

OSError: tout problème relié à un fichier (existence, lecture, écriture):

```
try:
```

```
    traiter_fichier("/tp/tp1/tp1.py")
```

```
except OSError as e:
```

```
    print(f"Erreur: {e}")
```

TypeError: quand le type n'est pas le bon:

```
>>> uneListe = [1,2,3]
```

```
>>> uneListe[1.2] TypeError
```

KeyboardInterrupt: quand l'utilisateur tape Control-C ou Delete

EOFError: essayer de lire quand on est en fin de fichier:

```
# test.py
```

```
x = input()
```

```
y = input()
```

```
$ echo "1 2" | python test.py
```

ZeroDivisionError: division par zéro

OverflowError: quand le résultat d'une opération mathématique est trop grand

Exemple: convertir une liste en nombres

```
1 #exceptionnel.py
2 DIGIT_MAP = {
3     'zéro': '0',
4     'un': '1',
5     'deux': '2',
6     'trois': '3',
7     'quatre': '4',
8     'cinq': '5',
9     'six': '6',
10    'sept': '7',
11    'huit': '8',
12    'neuf': '9',
13 }
14 #converti une liste ["un", "deux", "trois"] en nombre entier 123
15 def convertir(nombres):
16     nombre = ''
17     for s in nombres:
18         nombre += DIGIT_MAP[s]
19     x = int(nombre)
20     return x
21
22
```

```
>>> from exceptionnel import convertir
>>> convertir ("un trois trois sept".split())
1337

>>> convertir ("un million sept".split())
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "C:\Users\Michel\Downloads\scripts\exceptionnel.py", line 18, in convertir
    nombre += DIGIT_MAP[s]
KeyError: 'million'

>>>
```

Exemple: convertir une liste en nombres (1)

```
1 #exceptionnell1.py
2 DIGIT_MAP = {
3     'zéro': '0',
4     'un': '1',
5     'deux': '2',
6     'trois': '3',
7     'quatre': '4',
8     'cinq': '5',
9     'six': '6',
10    'sept': '7',
11    'huit': '8',
12    'neuf': '9',
13 }
14 #converti une liste ["un", "deux", "trois"] en nombre entier 123
15 def convertir(nombres):
16     try:
17         nombre = ''
18         for s in nombres:
19             nombre += DIGIT_MAP[s]
20         x = int(nombre)
21         print(f"Conversion réussie! x = {x}")
22     except KeyError:
23         print(f"Conversion manquée!")
24         x = -1
25     return x
```

```
>>> from exceptionnell1 import convertir
>>> convertir ("un trois trois sept".split())
    Conversion réussie! x = 1337
1337
>>> convertir ("un million sept".split())
    Conversion manquée!
-1
>>> convertir(512)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "C:\Users\Michel\Downloads\scripts\exceptionnell.py", line 18, in convertir
    for s in nombres:
TypeError: 'int' object is not iterable
>>>
```

Exemple: convertir une liste en nombres (2)

```
1 #exceptionnel2.py
2 DIGIT_MAP = {
3     'zéro': '0',
4     'un': '1',
5     'deux': '2',
6     'trois': '3',
7     'quatre': '4',
8     'cinq': '5',
9     'six': '6',
10    'sept': '7',
11    'huit': '8',
12    'neuf': '9',
13 }
14 #converti une liste ["un", "deux", "trois"] en nombre entier 123
15 def convertir(nombres):
16     try:
17         nombre = ''
18         for s in nombres:
19             nombre += DIGIT_MAP[s]
20         x = int(nombre)
21         print(f"Conversion réussie! x = {x}")
22     except KeyError:
23         print("Mauvaise clé!")
24         x = -1
25     except TypeError:
26         print("Mauvais type!")
27         x = -1
28     return x
```

```
>>> from exceptionnel2 import convertir
>>> convertir(512)

Mauvais type!
-1

>>>
```


Exemple: convertir une liste en nombres (3)

```
1 #exceptionnel3.py
2 DIGIT_MAP = {
3     'zéro': '0',
4     'un': '1',
5     'deux': '2',
6     'trois': '3',
7     'quatre': '4',
8     'cinq': '5',
9     'six': '6',
10    'sept': '7',
11    'huit': '8',
12    'neuf': '9',
13 }
14 #converti une liste ["un", "deux", "trois"] en nombre entier 123
15 def convertir(nombres):
16     x = -1
17     try:
18         nombre = ''
19         for s in nombres:
20             nombre += DIGIT_MAP[s]
21         x = int(nombre)
22         print(f"Conversion réussie! x = {x}")
23     except (KeyError, TypeError):
24         print("Conversion manquée!")
25     return x
```

```
>>> from exceptionnel3 import convertir
>>> convertir(512)

    Conversion manquée!
-1

>>> convertir ("un million sept".split())

    Conversion manquée!
-1

>>>
```

Exemple: convertir une liste en nombres (4)

```
1 #exceptionnel4.py
2 DIGIT_MAP = {
3     'zéro': '0',
4     'un': '1',
5     'deux': '2',
6     'trois': '3',
7     'quatre': '4',
8     'cinq': '5',
9     'six': '6',
10    'sept': '7',
11    'huit': '8',
12    'neuf': '9',
13 }
14 #converti une liste ["un", "deux", "trois"] en nombre entier 123
15 def convertir(nombres):
16     x = -1
17     try:
18         nombre = ''
19         for s in nombres:
20             nombre += DIGIT_MAP[s]
21         x = int(nombre)
22     except (KeyError, TypeError, IndentationError):
23
24     return x
```

```
1 # exceptionnel4x.py
2 try :
3     import exceptionnel4
4 except(IndentationError):
5     print("Mauvaise indentation!")
6
```

Console ✕

```
>>> %Run exceptionnel4x.py
Mauvaise indentation!
>>>
```

C'est une exception résultant du programmeur,.
On n'intercepte généralement pas ces exceptions (IndentationError, SyntaxError et NameError). Ici, il suffit d'ajouter **pass** à la ligne 23 pour éviter l'exception.

Exemple: convertir une liste en nombres (5)

```
1 #exceptionnel5.py
2 import sys
3 DIGIT_MAP = {
4     'zéro': '0',
5     'un': '1',
6     'deux': '2',
7     'trois': '3',
8     'quatre': '4',
9     'cinq': '5',
10    'six': '6',
11    'sept': '7',
12    'huit': '8',
13    'neuf': '9',
14 }
15 #converti une liste ["un", "deux", "trois"] en nombre entier 123
16 def convertir(nombres):
17     try:
18         nombre = ''
19         for s in nombres:
20             nombre += DIGIT_MAP[s]
21         return(int(nombre))
22     except (KeyError, TypeError) as e:
23         print(f"Conversion manquée: {repr(e)}", file = sys.stderr)
24         #repr(objet) retourne la version imprimable de cet objet
25         #repr() est généralement redéfinie pour cet objet
26         return -1
27     return x
```

```
>>> from exceptionnel5 import convertir
>>> convertir("un million sept".split())
    Conversion manquée: KeyError('million')
-1
>>> convertir(512)
    Conversion manquée: TypeError("'int' object is not iterable")
-1
>>>
```

Exemple: convertir une liste en nombres (6)

```
1 #exceptionnel6.py
2 import sys
3 from math import log
4 DIGIT_MAP = {
5     'zéro': '0',
6     'un': '1',
7     'deux': '2',
8     'trois': '3',
9     'quatre': '4',
10    'cinq': '5',
11    'six': '6',
12    'sept': '7',
13    'huit': '8',
14    'neuf': '9',
15 }
16 #converti une liste ["un", "deux", "trois"] en nombre entier 123
17 def convertir(nombres):
18     try:
19         nombre = ''
20         for s in nombres:
21             nombre += DIGIT_MAP[s]
22         return(int(nombre))
23     except (KeyError, TypeError) as e:
24         print(f"Conversion manquée: {repr(e)}", file = sys.stderr)
25         return -1
26     return x
27 #converti une liste ["un deux trois"] en son logarithme naturel 4.812184355372417
28 def string_log(s):
29     v = convertir(s)
30     return log(v)
```

```
>>> from exceptionnel6 import *
>>> string_log("zillion".split())

Conversion manquée: KeyError('zillion')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "C:\Users\Michel\Downloads\scripts\exceptionnel6.py", line 30, in string_log
    return log(v)
ValueError: math domain error

>>> log(-1)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: math domain error

>>> |
```

Exemple: convertir une liste en nombres (7)

```
1 #exceptionnel7.py
2 import sys
3 from math import log
4 DIGIT_MAP = {
5     'zéro': '0',
6     'un': '1',
7     'deux': '2',
8     'trois': '3',
9     'quatre': '4',
10    'cinq': '5',
11    'six': '6',
12    'sept': '7',
13    'huit': '8',
14    'neuf': '9',
15 }
16 #converti une liste ["un", "deux", "trois"] en nombre entier 123
17 def convertir(nombres):
18     try:
19         nombre = ''
20         for s in nombres:
21             nombre += DIGIT_MAP[s]
22         return(int(nombre))
23     except (KeyError, TypeError) as e:
24         raise # relance l'exception
25     return x
26 #converti une liste ["un deux trois"] en son logarithme naturel 4.812184355372417
27 def string_log(s):
28     try:
29         v = convertir(s)
30     except:
31         return -1
32     return log(v)
```

```
>>> %Run exceptionnel7.py
>>> from exceptionnel7 import *
>>> string_log("un two deux".split())
-1
>>> string_log("un trois deux".split())
4.882801922586371
>>>
```

Autre exemple: racine carrée d'un nombre

```
1 #heron.py
2 def sqrt(x):
3     """Calcul de la racine carrée avec la méthode d'Héron d'Alexandrie.
4     Argument:
5         x: le nombre à traiter.
6     Retourne:
7         La racine carrée de x.
8     """
9     approximation = x
10    i = 0
11    while approximation * approximation != x and i < 20:
12        approximation = (approximation + x / approximation) / 2.0
13        i += 1
14    return approximation
15 def main():
16     print(sqrt(9))
17     print(sqrt(2))
18     print(sqrt(-1))
19 if __name__ == '__main__':
20     main()
21
```

Console ✕

```
>>> %Run heron.py
```

```
3.0
```

```
1.414213562373095
```

```
Traceback (most recent call last):
```

```
File "C:\Users\Michel\OneDrive - Cégep André-Laurendeau\2021-A\555\getting-started-python-cor
main()
```

```
File "C:\Users\Michel\OneDrive - Cégep André-Laurendeau\2021-A\555\getting-started-python-cor
print(sqrt(-1))
```

```
File "C:\Users\Michel\OneDrive - Cégep André-Laurendeau\2021-A\555\getting-started-python-cor
approximation = (approximation + x / approximation) / 2.0
```

```
ZeroDivisionError: float division by zero
```

Autre exemple: racine carrée d'un nombre (1)

```
1 #heron1.py
2 import sys
3 def sqrt(x):
4     """Calcul de la racine carrée avec la méthode d'Héron d'Alexandrie.
5     Argument:
6         x: le nombre à traiter.
7     Retourne:
8         La racine carrée de x.
9     Exceptions levées:
10        ValueError: Si x est négatif.
11    """
12    if x < 0:
13        raise ValueError(f"Impossible de calculer la racine carrée de {x}")
14    approximation = x
15    i = 0
16    while approximation * approximation != x and i < 20:
17        approximation = (approximation + x / approximation) / 2.0
18        i += 1
19    return approximation
20 def main():
21     try:
22         print(sqrt(9))
23         print(sqrt(2))
24         print(sqrt(-1))
25         print("Ligne impossible à atteindre")
26     except ValueError as e:
27         print(e, file=sys.stderr)
28     print("L'exécution se poursuit ici.")
29 if __name__ == '__main__':
30     main()
```

```
>>> %Run heron1.py
3.0
1.414213562373095
Impossible de calculer la racine carrée de -1
L'exécution se poursuit ici.
>>>
```

Gestion des exceptions avec *finally*

```
1 #finalement.py
2 import os
3 import sys
4
5 def creer_dossier(chemin, dossier):
6     chemin_original = os.getcwd()
7     print(chemin_original)
8     try:
9         os.chdir(chemin)
10        os.mkdir(dossier)
11    except OSError as e:
12        print(e)
13    finally: # s'exécute indépendamment du résultat du bloc try
14        os.chdir(chemin_original)
15        print("retour au bercail de toute façon")
```

Console ✕

```
>>> %Run finalement.py
>>> creer_dossier("chemin", "fichier")

C:\Users\Michel\Downloads\scripts
[WinError 2] Le fichier spécifié est introuvable: 'chemin'
retour au bercail de toute façon

>>> creer_dossier("chemin", "fichier")

C:\Users\Michel\Downloads\scripts
retour au bercail de toute façon

>>>
```

en supposant ici qu'il
n'y a pas d'exception

Sommaire

- Une exception est un objet Python qui représente une erreur.
- Lorsqu'une erreur est rencontrée dans un programme, l'interpréteur Python lève une exception.
- Les erreurs de syntaxe sont également traitées comme des exceptions, mais généralement non gérées/traitées.
- Lorsqu'une exception se produit pendant l'exécution d'un programme et qu'une exception intégrée est définie pour cela, le code écrit dans cette exception s'exécute.
- Le processus de gestion des exceptions implique l'écriture de code supplémentaire pour donner des messages ou des instructions appropriés à l'utilisateur. Cela empêche le programme de terminer brusquement.
- Lever une exception implique d'interrompre le flux normal de l'exécution du programme et de sauter au gestionnaire d'exceptions.
- Les instructions à l'intérieur du bloc *finally* sont toujours exécutées, qu'une exception se produise ou non dans le bloc *try*.